

LaWAM: Latent World Action Models for Efficient Dynamics-Aware Robot Policies

Jialei Chen^{◆,▽,★} Kai Wang^{▶,▽} Kang Chen^{◆,▽} Shuaihang Chen^{■,▽} Feng Gao^{◆,★} Wenhao Tang[◆]
 Zhiyuan Li[◇] Weilin Liu[◇] Zhuyu Yao[◇] Boxun Li[◇] Yuanbo Xu^{◆†} Chao Yu^{◆†}
[◆]Tsinghua University [◇]Jilin University [▶]Nankai University [◆]Peking University
[■]Harbin Institute of Technology [▽]Zhongguancun Academy [★]Striding.AI [◇]Infinigence AI

Abstract: Vision-Language-Action models (VLAs) leverage large-scale vision-language pretraining for semantic robot control, but often lack explicit foresight into how robot actions change the scene. World-Action Models (WAMs) address this limitation by **conditioning policies on predicted futures**, yet existing approaches typically rely on computationally expensive video generation with substantial pixel-level redundancy. We present **LaWAM**, a **Latent World Action Model** that exposes predictive dynamics to robot policies through **compact latent visual subgoals** instead of reconstructed future video. At the core of LaWAM is a latent-action-conditioned **Latent World Model (LaWM)**. We obtain LaWM by training a latent action model in the latent space of a pretrained vision foundation model and repurposing its forward decoder to predict future observation features for scene evolution. LaWAM then conditions action generation on these **predicted latent visual subgoals** to enable dynamics-aware robot control. LaWAM achieves state-of-the-art or competitive success rates (SRs) across LIBERO (98.6% SR), RoboTwin (91.22% SR), and real-world manipulation tasks while retaining low-latency inference. LaWAM runs in 187 ms per action-chunk prediction and achieves up to 24× lower wall-clock latency than pixel-space WAMs.

Keywords: Robot Manipulation, World Action Models, Latent World Models

1 Introduction

Vision-Language-Action models (VLAs) [1, 2, 3, 4, 5] have recently shown strong performance on robotic manipulation by transferring large-scale vision-language pretraining into action generation. However, most current VLAs predict actions primarily from the current visual-language context, without explicitly modeling how the scene evolves under candidate actions [6, 7].

World-Action Models (WAMs) [8, 9, 10, 11, 12, 13, 14] offer a natural way to introduce temporal dynamics by augmenting policies with predicted future observations or states as additional context. However, current WAMs remain inefficient for manipulation policies. First, many methods predict future images or videos, allocating substantial modeling capacity to pixel-level synthesis rather than compact action-relevant dynamics. Second, iterative future generation introduces considerable inference latency; under the same evaluation setup used in Fig. 1, LingBot-VA [12] requires 4482 ms for a single policy inference, whereas the representative VLA $\pi_{0.5}$ [15] requires only 220 ms. Third, effective future prediction for manipulation should

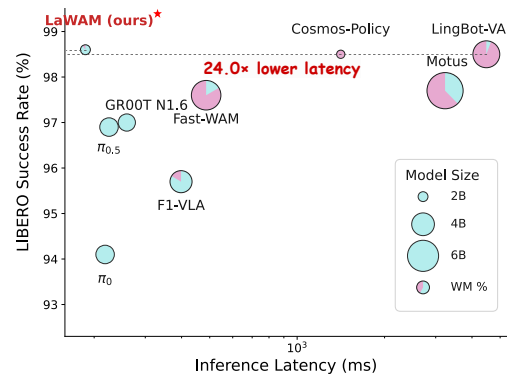


Figure 1: **Latency-success trade-off on LIBERO.** Latency for 10 denoising steps on an A100 GPU versus LIBERO success rate. The marker area denotes model size; the pink sector denotes world-modeling parameters.

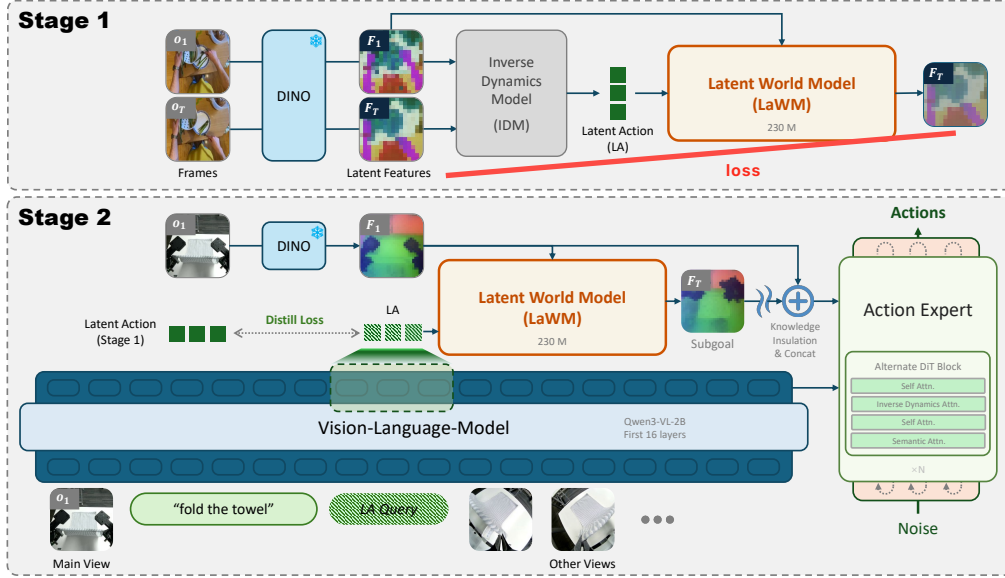


Figure 2: **Overview of LaWM.** Stage 1 learns a latent-action-conditioned **world model** from visual transitions: an **inverse-dynamics encoder** infers latent actions, and the decoder is retained as LaWM to predict future observation features. Stage 2 integrates LaWM into a VLA policy: **latent-action distillation teaches the policy to drive LaWM**, whose predicted latent visual subgoal is passed to an Alternate-DiT action expert for subgoal-conditioned action generation.

expose the state change relevant to the next action chunk, rather than merely generating visually plausible task-consistent futures.

To address these limitations, we propose a **Latent World Model (LaWM)** for WAMs that predicts compact latent visual subgoals directly, rather than synthesizing future images or videos. LaWM is a latent-action-conditioned dynamics model that predicts future observation features corresponding to the scene change required for the next action chunk. By operating entirely in the latent space, LaWM models action-relevant dynamics efficiently without expensive or iterative pixel generation.

We obtain LaWM by revisiting latent action models (**LAMs**) in the latent space of frozen visual encoders [16], such as DINOv3 [17]. A LAM contains an **inverse-dynamics encoder** that infers latent actions from visual transitions and a decoder that predicts the future latent state conditioned on the current latent state and latent action. Prior latent-action-based VLAs mainly use this framework to learn embodiment-agnostic action representations, treating the decoder only as an auxiliary training component and discarding it after pretraining [18, 16, 19, 20]. In contrast, we repurpose the decoder as the core predictive module of LaWM, using latent actions to generate future observation features that serve as latent visual subgoals for action generation.

Building on this latent visual subgoal interface, we introduce the **Latent World Action Model (LaWAM)**, which conditions action generation on LaWM-predicted future dynamics. Fig. 2 provides an overview of the full two-stage pretraining pipeline. We **first train LaWM as a latent-action-conditioned world model, then pretrain LaWAM with latent-action distillation for subgoal-conditioned action generation**. Overall, the training pipeline uses roughly 3,000 hours of robot videos and 1,500 hours of egocentric human videos. At test time, the policy predicts a latent action, LaWM decodes it into a latent visual subgoal in a single forward pass, and an action expert generates the action chunk conditioned on both the current context and predicted subgoal.

After benchmark-specific post-training, LaWAM achieves state-of-the-art or competitive success against strong VLA and WAM baselines across simulated benchmarks and physical robot tasks, including LIBERO [21], RoboTwin [22], and real-world pick-and-place, drawer opening, and towel folding, with 2.3B parameters. Beyond accuracy, Fig. 1 shows that LaWAM combines non-iterative latent prediction with a compact 230M-parameter LaWM, using about 95% fewer world-modeling

parameters than the 5B WAN backbone [23]. LaWAM runs in 187 ms per action-chunk prediction and achieves up to 24× lower wall-clock latency than pixel-space WAMs.

2 Related Work

Vision-Language-Action Models. VLAs transfer large-scale vision-language pretraining into robot control, giving policies strong semantic grounding over instructions, objects, and task compositions [1, 2, 4, 5, 14, 6]. This semantic prior helps specify *what* the robot should achieve, but it does not by itself provide explicit reasoning about *how* the scene evolves under embodied interaction. Recent VLA variants introduce temporal structure through learned future queries or feature-alignment objectives [24, 5, 25, 26]. These mechanisms are efficient and easy to integrate into VLA backbones, but they typically encode future dynamics into compact latent tokens rather than exposing an explicit action-conditioned future observation feature to the downstream action generator. LaWAM retains the semantic strengths of VLA backbones while introducing a latent world-model interface: a spatially structured latent visual subgoal that conditions action generation on predicted dynamics without reconstructing pixels.

World-Action Models. World-Action Models (WAMs) augment robot policies with predicted future context, allowing action generation to condition on how the scene may evolve under embodied interaction [8, 10, 13, 9, 12, 11, 27]. Pixel-space WAMs provide explicit physical foresight through generated images or videos, but they inherit costly iterative generation and substantial appearance-level redundancy [9, 12, 28]. $\pi_{0.7}$ similarly conditions robot policies on visual subgoals, but these subgoals are produced by a separate iterative pixel-space model [29]. Efficiency-oriented approaches such as Fast-WAM and Giga-World Policy reduce generation cost by making future prediction auxiliary or action-centered, which weakens the role of predicted dynamics as an explicit conditioning signal during test-time action generation [27, 11]. Concurrent analyses likewise suggest that pretrained visual latent spaces provide stronger policy-facing dynamics representations than pixel-reconstruction spaces [30]. While LDA-1B also models dynamics in a structured DINO latent space, it jointly denoises future visual states and action chunks through a diffusion-style process [31]. In contrast, LaWAM represents future dynamics through a single non-iterative latent visual subgoal that directly conditions the downstream action generator during inference.

Latent Action Models. Latent action models learn compact transition variables from unlabeled videos through latent inverse dynamics and forward prediction [18, 16, 19, 20]. Recent VLA and world-modeling methods use such variables as cross-embodiment action representations for policy learning and future prediction [19, 32, 33, 34, 35]. Most prior work focuses primarily on the latent action space itself—its discreteness or continuity, regularization, and downstream transfer. A complementary opportunity lies in the decoder: as concurrently observed by Garrido et al. [36], the decoder of a latent action model already implements a latent action-conditioned world model. LaWAM builds on this observation by repurposing the decoder as a policy-facing latent dynamics interface, where latent actions are expanded into embodiment-grounded future observation features that directly condition action generation.

3 Method

3.1 Problem Formulation

Let o denote the current observation, l the task instruction, and $a_{1:T}$ an action chunk over a fixed physical horizon τ . A standard VLA directly models $p(a_{1:T} | o, l)$, mapping the current perceptual and language context to executable actions. WAMs instead expose the policy to a predicted future. Using the horizon observation o_T for notation, a common WAM decomposition is

$$\underbrace{p(a_{1:T}, o_T | o, l)}_{\text{Joint}} = \underbrace{p(o_T | o, l)}_{\text{Future Prediction}} \underbrace{p(a_{1:T} | o, o_T)}_{\text{IDM}}. \quad (1)$$

reasonable

The second term acts as an inverse-dynamics model (**IDM**), mapping a desired visual future to actions. The same decomposition applies to future clips $o_{1:T}$. In either case, pixel-space WAMs must generate dense future images or videos, although chunk-level control often needs only a compact description of the relevant scene change.

LaWAM keeps the future-conditioned structure but represents the future in a frozen visual feature space. Let f_ψ be the frozen encoder, and define $u = f_\psi(o)$ and $u_T = f_\psi(o_T)$. We first learn a **latent action model over such feature pairs**:

$$z \sim q_\phi(z \mid u, u_T), \quad \tilde{u}_T = \text{LaWM}_\omega(u, z). \quad (2)$$

The distribution q_ϕ is the **latent-action posterior**: it acts as a latent inverse-dynamics model, inferring z from the observed transition (u, u_T) . The decoder then predicts the horizon feature from the current feature and z , and we retain this decoder as the Latent World Model (LaWM).

At inference time, the future feature u_T is **unavailable**, so the policy must predict the latent action before LaWM can predict the subgoal. LaWAM therefore, factors action generation as

$$\underbrace{p(a_{1:T}, \hat{u}_T, \hat{z} \mid o, l)}_{\text{LaWAM}} = \underbrace{p_\theta(\hat{z} \mid o, l)}_{\text{Policy Prior}} \underbrace{p_\omega(\hat{u}_T \mid u, \hat{z})}_{\text{LaWM}} \underbrace{p_\eta(a_{1:T} \mid o, l, u, \hat{u}_T)}_{\text{Action Expert}}. \quad (3)$$

Here p_θ predicts the latent action, p_ω deterministically decodes it into $\hat{u}_T = \text{LaWM}_\omega(u, \hat{z})$, and p_η generates the action chunk. We use u_T for the training target and $\hat{u}_T$ for the policy-driven subgoal.

Stage1: LaWM

3.2 Latent World Model

The first stage learns LaWM as the **forward decoder** of a latent action model. Each training example contains a current observation o and a horizon observation o_T sampled after the physical interval τ . After encoding them as (u, u_T) , the inverse-dynamics encoder infers a latent action $z \sim q_\phi(z \mid u, u_T)$. The decoder then uses (u, z) to predict the horizon feature, $\tilde{u}_T = \text{LaWM}_\omega(u, z)$.

This training setup gives LaWM a simple role: given the current latent state and a latent action, predict the latent future state. The target u_T directly supervises the decoder, and the inferred latent action z serves as the teacher signal for the stage-two policy prior.

We add one **auxiliary signal** during this pretraining stage. A lightweight predictor g maps the current end-effector state s and latent action z to the horizon state s_T , **encouraging z to encode embodied motion rather than only visual appearance change**. After pretraining, we **discard** this auxiliary head and keep the decoder as LaWM; the posterior encoder is used only to produce teacher latent actions for policy training. LaWM architecture details are provided in Appendix C.1.

We train the latent action model with a forward-prediction objective and **KL regularization**,

$$\mathcal{L}_{\text{LAM}} = \mathcal{L}_{\text{wm}} + \mathcal{L}_{\text{aux}} + \beta D_{\text{KL}}(q_\phi(z \mid u, u_T) \parallel \mathcal{N}(0, I)), \quad (4)$$

where $\mathcal{L}_{\text{wm}} = \|\tilde{u}_T - u_T\|_2^2$ trains LaWM to match the horizon feature, and $\mathcal{L}_{\text{aux}} = \|g(s, z) - s_T\|_2^2$ trains the auxiliary state predictor. The KL term regularizes the latent-action space so that it can later be modeled by the policy prior. Because both u_T and s_T are defined by the same physical interval τ , the learned subgoal corresponds to a consistent amount of elapsed motion rather than a dataset-specific frame offset.

Stage2: LaWAM

3.3 Latent World Action Model

The second stage turns the pretrained LaWM into a test-time policy interface. The IDM encoder from stage one cannot be used during deployment, because it requires the future feature u_T . We therefore train the **policy prior** $p_\theta(\hat{z} \mid o, l)$ to predict the latent action from the current observation and instruction. The predicted latent action is passed through the pretrained LaWM decoder to produce $\hat{u}_T = \text{LaWM}_\omega(u, \hat{z})$, giving the policy a latent prediction of chunk-level visual future without generating future pixels.

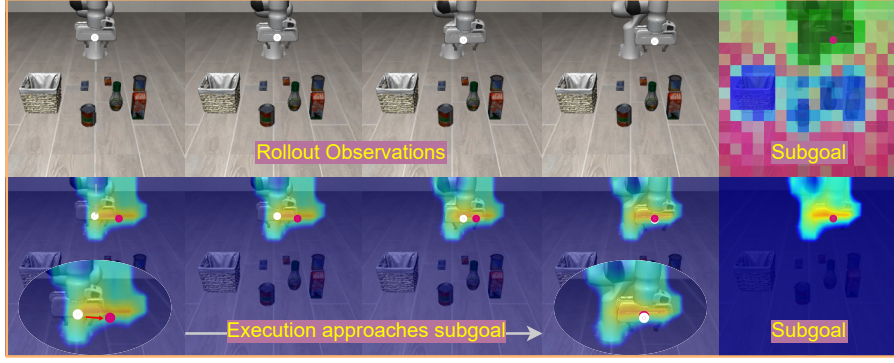


Figure 3: **Subgoal-guided chunk execution.** The top row shows observations within one executed LIBERO chunk together with the predicted latent subgoal; the bottom row overlays subgoal-derived robot-arm heatmaps, illustrating how the executed motion approaches the predicted subgoal region.

The action expert uses this predicted future through an Alternate-DiT design [5]. One stream carries the **semantic context** from the VLM backbone, while the other carries the **latent dynamics context** formed by $(u, \hat{u}_T)$. Alternating between these streams lets the expert combine task intent with the predicted scene evolution when denoising the action chunk. Fig. 3 visualizes the resulting chunk-level execution: the action expert generates a robot motion chunk conditioned on the predicted latent subgoal, and the executed motion moves toward the corresponding subgoal region. Backbone, query layout, and Alternate-DiT details are given in Appendix C.2.

Stage-two training combines ¹latent-action distillation, ²subgoal supervision, and ³action flow matching. We further apply Knowledge Insulation (KI) [37] so action-expert gradients do not overwrite the pretrained LaWM dynamics. For mixed-frequency robot data, LaWAM keeps each dataset at its native control frequency and uses physical-time encoding, as detailed in Appendix C.3.

The full stage-two objective is

$$\mathcal{L}_{\text{LaWAM}} = \lambda_{\text{distill}} \mathcal{L}_{\text{distill}} + \lambda_{\text{wm}} \mathcal{L}_{\text{wm}} + \mathcal{L}_{\text{act}}. \quad (5)$$

Here $\mathcal{L}_{\text{distill}} = \mathbb{E}[\|\hat{z} - z\|_2^2]$, $\mathcal{L}_{\text{wm}} = \|\hat{u}_T - u_T\|_2^2$ supervises the policy-driven subgoal, and $\mathcal{L}_{\text{act}}$ denotes the conditional flow-matching loss for $a_{1:T}$ given $(o, l, u, \hat{u}_T)$. Training schedules and optimization details are reported in Appendix C.5.

4 Experiments

We use a modest pretraining setup built from open-source data [38, 39, 40, 41, 42, 43, 44, 45]. LaWM is pretrained on roughly 3,000 hours of robot videos and 1,500 hours of egocentric human videos, while LaWAM policy integration uses only robot trajectories with language instructions; human videos contribute only through the dynamics prior learned by LaWM. We evaluate LaWAM along five axes: simulated benchmark performance, inference efficiency, real-world transfer, latent-dynamics behavior, and component contributions. Benchmark-specific protocols are detailed in Appendices D.1, D.2, and D.3, with visualization protocols in Appendix B.

4.1 LIBERO

We first evaluate on the four standard LIBERO suites [21]. As shown in Table 1, LaWAM achieves the best average success rate among the compared VLA, latent-action, and WAM baselines; the gap over latent-action baselines suggests that compact action tokens are more effective when expanded into spatially structured latent visual subgoals.

This performance does not require expensive pixel-space imagination. Representative pixel-space WAMs rely on large video-generation backbones, whereas LaWAM uses a 230M LaWM in place of the 5B WAN backbone used by such models, reducing world-modeling parameters by about 95%. This keeps the full LaWAM model size at 2.3B and yields 187 ms latency, up to $24\times$ faster than

Table 1: **LIBERO benchmark results over 50 trials per task.** Latency is model-only wall-clock time per action chunk, excluding simulator and robot execution overhead. The best result in each metric is shown in bold, the second-best is underlined, and large models or high latency are highlighted in red.

Method	Model Size	Latency (ms)	Long	Goal	Object	Spatial	Average
MAINSTREAM VLA							
OpenVLA-OFT [2]	7B	—	94.5	97.9	98.4	97.6	97.1
π_0 [3]	3.5B	<u>220</u>	88.4	94.4	96.8	98.0	94.4
$\pi_{0.5}$ [15]	3.5B	<u>220</u>	92.4	98.0	98.2	<u>98.8</u>	96.9
GR00T-N1.6 [5]	3.3B	259	94.4	97.5	98.5	97.7	97.0
LATENT ACTION BASED METHODS							
LAPA [19]	7B	—	55.4	58.8	74.6	73.8	65.7
UniVLA [32]	7B	—	92.0	95.6	96.8	96.5	95.2
Mantis [6]	5.8B	—	94.2	94.4	99.2	<u>98.8</u>	96.7
VLA-JEPA [33]	3B	—	95.8	97.2	99.6	96.2	97.2
WAMS IN PIXEL SPACE							
F1 [14]	4B	399.0	91.3	95.4	97.8	98.2	95.7
Motus [10]	8B	3231	<u>97.6</u>	96.6	<u>99.8</u>	96.8	97.7
Cosmos-Policy [9]	2.1B	1413	<u>97.6</u>	<u>98.2</u>	100.0	98.1	<u>98.5</u>
LingBot-VA [12]	5.5B	4482	98.5	97.2	99.6	98.5	<u>98.5</u>
Fast-WAM [27]	6B	486	95.2	97.0	100.0	98.2	97.6
LATENT WORLD ACTION MODEL							
LaWAM	2.3B	187	97.0	98.4	99.6	99.4	98.6

Table 2: **RoboTwin benchmark results over 100 trials per task.** Fast-WAM and LingBot-VA are re-evaluated from their open weights on H100 GPUs; the remaining baseline results are taken from Fast-WAM [27] and GigaWorld-Policy [11]. Best results are marked in bold.

Task	Fast-WAM		GigaWorld-Policy		LingBot-VA		$\pi_{0.5}$		Motus		LaWAM	
	Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.
Move Can Pot	95	92	76	78	93	93	51	55	34	74	98	93
Move Stapler Pad	84	63	92	82	59	71	56	42	83	85	94	87
Open Laptop	100	100	96	98	96	90	90	96	95	91	100	100
Pick Dual Bottles	100	94	86	86	100	100	93	63	96	90	100	95
Place A2B Left	94	89	94	88	96	94	87	82	88	79	98	91
... (50 tasks in total)												
Place Container Plate	98	100	98	96	97	98	99	95	98	99	100	100
Place Dual Shoes	88	88	96	84	94	81	75	75	93	87	98	94
Place Object Basket	82	90	90	92	89	85	80	76	81	87	92	90
Place Object Scale	83	88	88	80	98	87	86	80	88	85	95	88
Place Shoe	97	98	98	96	99	97	92	93	99	97	100	100
Put Bottles Dustbin	93	82	72	70	82	87	84	79	81	79	94	92
Put Object Cabinet	94	82	74	74	87	80	80	79	88	71	90	82
Scan Object	96	86	60	64	90	89	72	65	67	66	96	90
Stack Bowls Two	90	96	96	92	98	99	95	96	98	98	100	99
Average	91.98	90.52	86.36	85.04	91.50	90.92	82.74	76.76	88.66	87.02	92.64	89.80

pixel-space WAMs as shown in Fig. 1. The efficiency comes from predicting one latent subgoal per action chunk rather than repeatedly generating pixel-space future frames. Appendix Fig. 8 extends the chunk-level visualization in Fig. 3 to complete executed LIBERO trajectories.

4.2 RoboTwin

RoboTwin 2.0 [22] evaluates coordinated bimanual manipulation across 50 tasks, allowing us to test whether non-iterative latent world modeling scales beyond single-arm LIBERO tasks. Table 2 shows that LaWAM achieves the best clean-scene average and remains close to the strongest pixel-space WAMs under randomized scenes. These results suggest that latent subgoal prediction scales to complex bimanual manipulation while avoiding heavy test-time rollout cost of pixel-space WAMs.

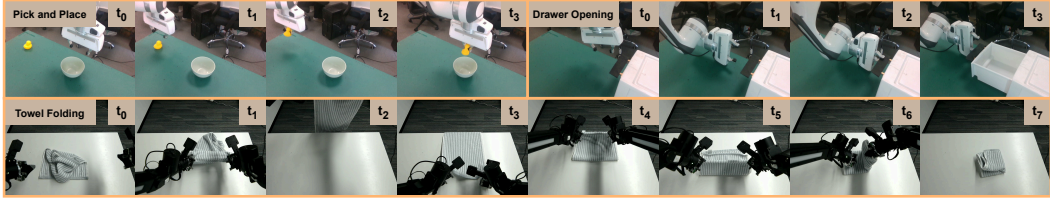


Figure 4: Representative real-world rollouts for pick-and-place, drawer opening, and towel folding on two robot platforms.

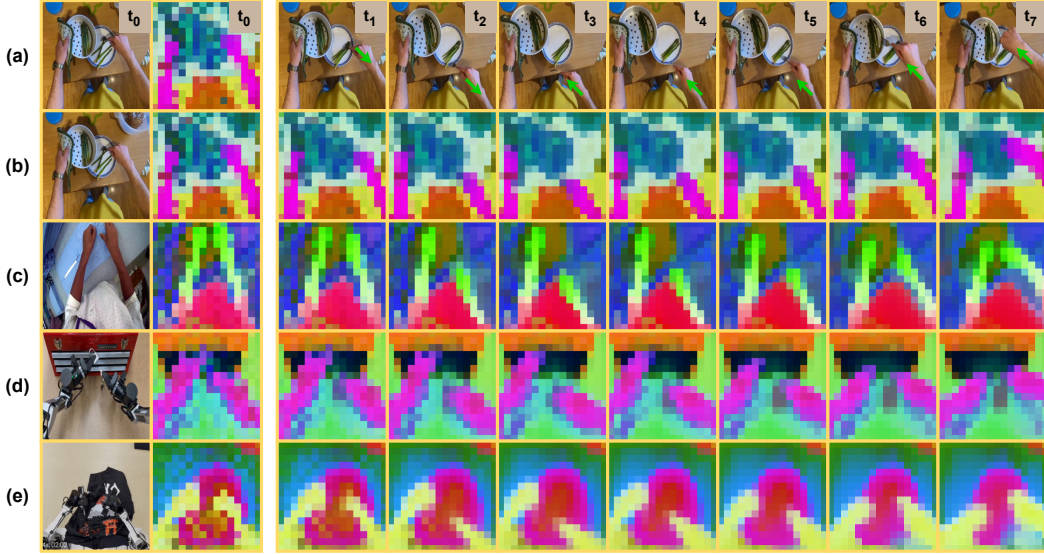


Figure 5: **Cross-embodiment open-loop LaWM rollouts from shared latent actions.** (a) We infer a latent-action trajectory from the source video. (b)–(e) Given only one initial observation in each environment or embodiment, LaWM applies the same extracted latent actions to generate context-specific latent rollouts. Panels (d) and (e) use unseen screenshots from *pi.website*.

4.3 Real-World Experiments

For real-world transfer, we evaluate Pick-and-Place, Drawer Opening, and Towel Folding, covering rigid-object manipulation, articulated-object interaction, and long-horizon deformable-object manipulation. The test trials include both in-distribution initial configurations and out-of-distribution spatial configurations beyond the training demonstrations, testing whether latent dynamics learned from heterogeneous robot data transfer beyond benchmark environments. Fig. 4 shows representative real-world rollouts, and complete trajectory subgoal visualizations are provided in Appendix Figs. 11, 12, and 13.

Table 3: Real-world SR over 30 trials per task (%).

Method	Pick-and-Place	Open Drawer	Fold Towel	Avg.
$\pi_{0.5}$	86.7	80.0	83.3	83.3
GR00T-N1.6	83.3	76.7	46.7	68.9
Fast-WAM	56.7	63.3	70.0	63.3
LingBot-VA	76.7	83.3	0.0	53.3
LaWAM	93.3	86.7	90.0	90.0

Table 3 shows that LaWAM achieves the best average real-world success rate and ranks first across all three tasks. It completes tasks stably and controls spatial target positions accurately, indicating that dense LaWM features encode useful spatial structure for real-world control. The advantage is especially clear in towel folding, where successful execution requires timely responses to dynamic cloth motion; high-latency baselines such as LingBot-VA can pause while the towel continues moving, causing delayed actions to become mismatched to the current cloth state.

4.4 LaWM Dynamics Analysis

We next analyze whether LaWM captures coherent dynamics rather than merely providing an auxiliary feature for the policy. Open-loop analyses in Fig. 5 show that applying the same latent action to unseen environments and embodiments produces coherent latent-space changes. This cross-embodiment rollout suggests two complementary properties: the latent action captures an embodiment-agnostic visual transition, while LaWM grounds it in the current latent visual state, which retains the embodiment-specific information needed to realize the dynamics. This also clarifies why using latent actions alone as the final policy interface can be limiting: the latent action becomes most useful after LaWM expands it into a visual subgoal grounded in the current embodiment. Appendix D.4 provides additional open-loop rollout and cross-embodiment visualizations of LaWM’s dynamics-modeling behavior.

4.5 Component Ablations

We conduct LIBERO ablations to isolate the contribution of each part of the LaWAM interface. As shown in Fig. 6, performance degrades as this interface is progressively weakened. Removing LaWM causes the largest drop, especially on LIBERO-Long, showing that explicit latent subgoal conditioning is the main source of the gain. Removing latent-action distillation also substantially hurts performance, indicating that the policy needs direct supervision from the LAM posterior to reliably drive LaWM. The combined *w/o KI & distill* variant degrades further, suggesting that LaWM should be both driven by aligned latent actions and protected from action-expert gradients during policy learning.

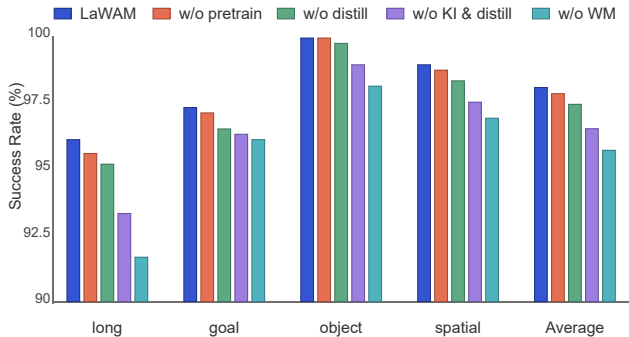


Figure 6: **Component ablations on LIBERO.** The results show the contribution of pretraining, latent-action distillation, knowledge insulation, and LaWM.

5 Limitations

LaWAM is currently most effective in manipulation settings with relatively stable camera views. When camera motion dominates the observed transition, as in egocentric videos with abrupt shake or large viewpoint changes, LaWM can fail to learn a coherent latent action space. This limits the present formulation for humanoid or mobile robots whose observations are strongly shaped by self-motion. A second limitation is data coverage: fine-grained deformable-object dynamics, such as subtle cloth deformation during towel folding, are rare in the current training mixture, making them harder for LaWM to model reliably. Future work will scale LaWAM with broader data and model capacity to improve robustness under moving cameras and finer-grained physical interactions.

6 Conclusion

We presented **LaWAM**, a latent World-Action Model that brings predictive dynamics into robot policy inference without reconstructing pixel-space futures. LaWAM repurposes the forward decoder of a latent action model as LaWM, which **expands policy-predicted latent actions into embodiment-grounded latent visual subgoals for action-chunk generation**. Across simulated and real-world manipulation tasks, these subgoals provide action-relevant future context while avoiding the latency and parameter cost of pixel-space WAMs, suggesting that future prediction can serve as a compact latent interface between semantic instruction following and physically grounded control.

References

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, and A. Balakrishna. Openvla: An open-source vision-language-action model. arXiv preprint arXiv:2406.09246, Sept. 2024. URL <http://arxiv.org/abs/2406.09246>.
- [2] M. J. Kim, C. Finn, and P. Liang. Openvla-ofit: Fine-tuning vision-language-action models: Optimizing speed and success. arXiv preprint arXiv:2502.19645, Apr. 2025. URL <http://arxiv.org/abs/2502.19645>.
- [3] K. Black, N. Brown, D. Driess, A. Esmail, and M. Equi. π_0 : A vision-language-action flow model for general robot control. arXiv preprint arXiv:2410.24164, Nov. 2024. URL <http://arxiv.org/abs/2410.24164>.
- [4] G. R. Team, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, and M. G. Arenas. Gemini robotics: Bringing ai into the physical world. arXiv preprint arXiv:2503.20020, Mar. 2025. URL <http://arxiv.org/abs/2503.20020>.
- [5] Nvidia, J. Bjorck, F. Castañeda, N. Cherniadev, and X. Da. Gr00t n1: An open foundation model for generalist humanoid robots. arXiv preprint arXiv:2503.14734, Mar. 2025. URL <http://arxiv.org/abs/2503.14734>.
- [6] Y. Yang, X. Li, Y. Chen, J. Song, and Y. Wang. Mantis: A versatile vision-language-action model with disentangled visual foresight. arXiv preprint arXiv:2511.16175, Nov. 2025. URL <http://arxiv.org/abs/2511.16175>.
- [7] Y. Chen, Y. Ge, H. Zhou, M. Ding, Y. Ge, and X. Liu. Dial: Decoupling intent and action via latent world modeling for end-to-end vla. arXiv preprint arXiv:2603.29844v1, Mar. 2026. URL <https://arxiv.org/abs/2603.29844v1>.
- [8] J. Pai, L. Achenbach, V. Montesinos, B. Forrai, O. Mees, and E. Nava. Mimic-video: Video-action models for generalizable robot control beyond vlas. arXiv preprint arXiv:2512.15692, Dec. 2025. URL <http://arxiv.org/abs/2512.15692>.
- [9] M. J. Kim, Y. Gao, T.-Y. Lin, Y.-C. Lin, and Y. Ge. Cosmos policy: Fine-tuning video models for visuomotor control and planning. arXiv preprint arXiv:2601.16163, Jan. 2026. URL <http://arxiv.org/abs/2601.16163>.
- [10] H. Bi, H. Tan, S. Xie, Z. Wang, and S. Huang. Motus: A unified latent action world model. arXiv preprint arXiv:2512.13030, Dec. 2025. URL <http://arxiv.org/abs/2512.13030>.
- [11] A. Ye, B. Wang, C. Ni, G. Huang, and G. Zhao. Gigaworld-policy: An efficient action-centered world-action model. arXiv preprint arXiv:2603.17240, Mar. 2026. URL <http://arxiv.org/abs/2603.17240>.
- [12] L. Li, Q. Zhang, Y. Luo, S. Yang, and R. Wang. Causal world modeling for robot control. arXiv preprint arXiv:2601.21998, Mar. 2026. URL <http://arxiv.org/abs/2601.21998>.
- [13] S. Ye, Y. Ge, K. Zheng, S. Gao, and S. Yu. World action models are zero-shot policies. arXiv preprint arXiv:2602.15922, Feb. 2026. URL <http://arxiv.org/abs/2602.15922>.
- [14] Q. Lv, W. Kong, H. Li, J. Zeng, and Z. Qiu. F1: A vision-language-action model bridging understanding and generation to actions. arXiv preprint arXiv:2509.06951, Sept. 2025. URL <http://arxiv.org/abs/2509.06951>.
- [15] P. Intelligence, K. Black, N. Brown, J. Darpinian, and K. Dhabalia. $\pi_{0.5}$: A vision-language-action model with open-world generalization. arXiv preprint arXiv:2504.16054, Apr. 2025. URL <http://arxiv.org/abs/2504.16054>.

- [16] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, and Y. Shi. Genie: Generative interactive environments. arXiv preprint arXiv:2402.15391, Feb. 2024. URL <http://arxiv.org/abs/2402.15391>.
- [17] O. Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, and M. Oquab. Dinov3. arXiv preprint arXiv:2508.10104, Aug. 2025. URL <http://arxiv.org/abs/2508.10104>.
- [18] D. Schmidt and M. Jiang. Learning to act without actions. arXiv preprint arXiv:2312.10812, Mar. 2024. URL <http://arxiv.org/abs/2312.10812>.
- [19] S. Ye, J. Jang, B. Jeon, S. Joo, and J. Yang. Lapa: Latent action pretraining from videos. arXiv preprint arXiv:2410.11758, May 2025. URL <http://arxiv.org/abs/2410.11758>.
- [20] J. Yang, Y. Shi, H. Zhu, M. Liu, and K. Ma. Como: Learning continuous latent motion from internet videos for scalable robot learning. arXiv preprint arXiv:2505.17006, May 2025. URL <http://arxiv.org/abs/2505.17006>.
- [21] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. arXiv preprint arXiv:2306.03310, June 2023. URL <https://arxiv.org/abs/2306.03310>.
- [22] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu, et al. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. arXiv preprint arXiv:2506.18088, June 2025. URL <https://arxiv.org/abs/2506.18088>.
- [23] T. Wan, A. Wang, B. Ai, B. Wen, and C. Mao. Wan: Open and advanced large-scale video generative models. arXiv preprint arXiv:2503.20314, Apr. 2025. URL <http://arxiv.org/abs/2503.20314>.
- [24] R. Zheng, J. Wang, S. Reed, J. Bjorck, and Y. Fang. Flare: Robot learning with implicit world modeling. arXiv preprint arXiv:2505.15659, May 2025. URL <http://arxiv.org/abs/2505.15659>.
- [25] B. Team. Being-h0.7: A latent world-action model from egocentric videos. arXiv preprint arXiv:2605.00078, 2026. URL <https://arxiv.org/abs/2605.00078>.
- [26] Y. Su, S. Chen, H. Shi, M. Liu, and Z. Zhang. World guidance: World modeling in condition space for action generation. arXiv preprint arXiv:2602.22010, Feb. 2026. URL <http://arxiv.org/abs/2602.22010>.
- [27] T. Yuan, Z. Dong, Y. Liu, and H. Zhao. Fast-wam: Do world action models need test-time future imagination? arXiv preprint arXiv:2603.16666, Mar. 2026. URL <http://arxiv.org/abs/2603.16666>.
- [28] Z. Jiang, S. Zhou, Y. Jiang, Z. Huang, M. Wei, Y. Chen, T. Zhou, Z. Guo, H. Lin, Q. Zhang, Y. Wang, H. Li, C. Yu, and D. Zhao. Wovr: World models as reliable simulators for post-training vla policies with rl. arXiv preprint arXiv:2602.13977, 2026. URL <https://arxiv.org/abs/2602.13977>.
- [29] Physical Intelligence. $\pi_{0.7}$: a Steerable Generalist Robotic Foundation Model with Emergent Capabilities. arXiv preprint arXiv:2604.15483, Apr. 2026. URL <https://arxiv.org/abs/2604.15483>.
- [30] Nilaksh, S. Jha, A. Zholus, and S. Chandar. Reconstruction or semantics? what makes a latent space useful for robotic world models. arXiv preprint arXiv:2605.06388, May 2026. URL <https://arxiv.org/abs/2605.06388>.

- [31] J. Lyu, K. Liu, X. Zhang, H. Liao, and Y. Feng. Lda-1b: Scaling latent dynamics action model via universal embodied data ingestion. arXiv preprint arXiv:2602.12215, Feb. 2026. URL <http://arxiv.org/abs/2602.12215>.
- [32] Q. Bu, Y. Yang, J. Cai, S. Gao, and G. Ren. Univla: Learning to act anywhere with task-centric latent actions. arXiv preprint arXiv:2505.06111, May 2025. URL <http://arxiv.org/abs/2505.06111>.
- [33] J. Sun, W. Zhang, Z. Qi, S. Ren, and Z. Liu. Vla-jepa: Enhancing vision-language-action model with latent world model. arXiv preprint arXiv:2602.10098, Feb. 2026. URL <http://arxiv.org/abs/2602.10098>.
- [34] S. Gao, S. Zhou, Y. Du, J. Zhang, and C. Gan. Adaworld: Learning adaptable world models with latent actions. arXiv preprint arXiv:2503.18938, June 2025. URL <http://arxiv.org/abs/2503.18938>.
- [35] S. Gao, W. Liang, K. Zheng, A. Malik, and S. Ye. Dreamdojo: A generalist robot world model from large-scale human videos. arXiv preprint arXiv:2602.06949, Feb. 2026. URL <http://arxiv.org/abs/2602.06949>.
- [36] Q. Garrido, T. Nagarajan, B. Terver, N. Ballas, Y. LeCun, and M. Rabbat. Learning latent action world models in the wild. arXiv preprint arXiv:2601.05230, Jan. 2026. URL <http://arxiv.org/abs/2601.05230>.
- [37] D. Driess, J. T. Springenberg, B. Ichter, L. Yu, and A. Li-Bell. Knowledge insulating vision-language-action models: Train fast, run fast, generalize better. arXiv preprint arXiv:2505.23705, May 2025. URL <http://arxiv.org/abs/2505.23705>.
- [38] R. Hoque, P. Huang, D. J. Yoon, M. Sivapurapu, and J. Zhang. Egodex: Learning dexterous manipulation from large-scale egocentric video. arXiv preprint arXiv:2505.11709, May 2025. URL <http://arxiv.org/abs/2505.11709>.
- [39] K. Grauman, A. Westbury, E. Byrne, Z. Chavis, and A. Furnari. Ego4d: Around the world in 3,000 hours of egocentric video. arXiv preprint arXiv:2110.07058, Mar. 2022. URL <http://arxiv.org/abs/2110.07058>.
- [40] B. Lai, X. Dai, L. Chen, G. Pang, J. M. Rehg, and M. Liu. Lego: Learning egocentric action frame generation via visual instruction tuning. arXiv preprint arXiv:2312.03849, Mar. 2024. URL <http://arxiv.org/abs/2312.03849>.
- [41] AgiBot-World-Contributors, Q. Bu, J. Cai, L. Chen, and X. Cui. Agibot world colosseum: A large-scale manipulation platform for scalable and intelligent embodied systems. arXiv preprint arXiv:2503.06669, Apr. 2025. URL <http://arxiv.org/abs/2503.06669>.
- [42] K. Wu, C. Hou, J. Liu, Z. Che, X. Ju, Z. Yang, M. Li, Y. Zhao, Z. Xu, G. Yang, et al. Robomind: Benchmark on multi-embodiment intelligence normative data for robot manipulation. In *Robotics: Science and Systems (RSS) 2025*. Robotics: Science and Systems Foundation, 2025. URL <https://www.roboticsproceedings.org/rss21/p152.pdf>.
- [43] S. Wu, X. Liu, S. Xie, et al. Robocoin: An open-sourced bimanual robotic data collection for integrated manipulation. arXiv preprint arXiv:2511.17441, 2025. URL <https://arxiv.org/abs/2511.17441>.
- [44] O. X.-E. Collaboration. Open X-Embodiment: Robotic learning datasets and RT-X models. arXiv preprint arXiv:2310.08864, 2023. URL <https://arxiv.org/abs/2310.08864>.
- [45] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, and S. Dasari. Droid: A large-scale in-the-wild robot manipulation dataset. arXiv preprint arXiv:2403.12945, Apr. 2025. URL <http://arxiv.org/abs/2403.12945>.

- [46] M. Assran, A. Bardes, D. Fan, Q. Garrido, and R. Howes. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. arXiv preprint arXiv:2506.09985, June 2025. URL <http://arxiv.org/abs/2506.09985>.
- [47] J. Zhao, W. Lu, D. Zhang, Y. Liu, and Y. Liang. Do you need proprioceptive states in visuomotor policies? arXiv preprint arXiv:2509.18644, Sept. 2025. URL <http://arxiv.org/abs/2509.18644>.

Table 4: Per-task success rates on RoboTwin under clean and randomized evaluation settings.

Task	Fast-WAM		GigaWorld-Policy		LingBot-VA		$\pi_{0.5}$		Motus		LaWAM	
	Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.
Adjust Bottle	100	100	100	100	98	99	100	99	89	93	100	100
Beat Block Hammer	99	100	86	86	96	98	96	93	95	88	90	93
Blocks Ranking RGB	100	100	92	96	98	96	92	85	99	97	97	100
Blocks Ranking Size	94	98	44	48	96	89	49	26	75	63	93	89
Click Alarmclock	100	100	100	100	100	99	98	89	100	100	100	100
Click Bell	100	100	100	100	100	100	99	66	100	100	100	100
Dump Bin Bigbin	95	93	92	100	87	95	92	97	95	91	97	95
Grab Roller	100	100	100	100	100	100	100	100	100	100	100	100
Handover Block	99	80	80	80	100	91	66	57	86	73	96	87
Handover Mic	100	100	72	72	96	95	98	97	78	63	93	98
Hanging Mug	65	56	16	12	21	33	18	17	38	38	51	43
Lift Pot	100	100	98	98	100	100	96	85	96	99	100	99
Move Can Pot	95	92	76	78	93	93	51	55	34	74	98	93
Move Pillbottle Pad	100	100	90	90	100	99	84	61	93	96	97	90
Move Playingcard Away	100	100	78	72	100	100	96	84	100	96	100	100
Move Stapler Pad	84	63	92	82	59	71	56	42	83	85	94	87
Open Laptop	100	100	96	98	96	90	90	96	95	91	100	100
Open Microwave	46	34	74	66	56	80	34	77	95	91	41	43
Pick Diverse Bottles	81	88	82	70	93	87	81	71	90	91	91	88
Pick Dual Bottles	100	94	86	86	100	100	93	63	96	90	100	95
Place A2B Left	94	89	94	88	96	94	87	82	88	79	98	91
Place A2B Right	98	90	90	92	91	94	87	84	91	87	89	94
Place Bread Basket	86	94	82	82	94	94	77	64	91	94	92	85
Place Bread Skillet	92	94	94	90	95	92	85	66	86	83	90	83
Place Burger Fries	98	92	98	96	95	98	94	87	98	98	93	96
Place Can Basket	72	67	78	74	83	80	62	62	81	76	92	65
Place Cans Plasticbox	98	100	100	100	100	98	94	84	98	94	100	95
Place Container Plate	98	100	98	96	97	98	99	95	98	99	100	100
Place Dual Shoes	88	88	96	84	94	81	75	75	93	87	98	94
Place Empty Cup	100	100	90	90	100	100	100	99	99	98	99	100
Place Fan	100	96	92	94	95	88	87	85	91	87	92	93
Place Mouse Pad	84	89	88	90	92	93	60	39	66	68	91	84
Place Object Basket	82	90	90	92	89	85	80	76	81	87	92	90
Place Object Scale	83	88	88	80	98	87	86	80	88	85	95	88
Place Object Stand	88	91	100	98	99	91	91	85	98	97	92	93
Place Phone Stand	98	98	82	72	98	97	81	81	87	86	93	94
Place Shoe	97	98	98	96	99	97	92	93	99	97	100	100
Press Stapler	98	100	96	96	82	88	87	83	93	98	98	97
Put Bottles Dustbin	93	82	72	70	82	87	84	79	81	79	94	92
Put Object Cabinet	94	82	74	74	87	80	80	79	88	71	90	82
Rotate QRcode	95	94	90	84	92	95	89	87	89	73	94	89
Scan Object	96	86	60	64	90	89	72	65	67	66	96	90
Shake Bottle	100	100	100	100	100	98	99	97	100	97	100	100
Shake Bottle Horizontally	100	100	100	98	99	99	99	99	100	98	100	100
Stack Blocks Three	98	96	70	78	100	99	91	76	91	95	90	75
Stack Blocks Two	100	100	100	94	100	99	97	100	100	98	100	97
Stack Bowls Three	77	86	70	72	89	79	77	71	79	87	90	80
Stack Bowls Two	90	96	96	92	98	99	95	96	98	98	100	99
Stamp Seal	78	86	96	98	98	95	79	55	93	92	89	88
Turn Switch	66	56	82	84	54	57	62	54	84	78	47	56
Average	91.98	90.52	86.36	85.04	91.50	90.92	82.74	76.76	88.66	87.02	92.64	89.80

A Detailed RoboTwin Results

B Qualitative Visualization Protocol

Figs. 3, 8, 11, 12, and 13, listed in the order they are introduced in the main text, visualize LaWAM subgoals and overlay them with the corresponding observations. Here we describe how these

visualizations are produced. We use the predicted LaWM subgoal directly in the latent DINO feature space rather than reconstructing future pixels.

For each sequence, we select a robot-arm patch in the initial observation, extract its DINO feature, and measure its cosine similarity to every patch in the predicted subgoal feature map. The green box in the first panel of Fig. 8 shows one selected patch. This heatmap reveals where the current robot-arm feature is expected to move in the latent subgoal and helps interpret the dynamics encoded by LaWM without requiring pixel-space reconstruction. During LaWAM inference, we consistently observe that within each action chunk, the executed robot motion drives the arm toward the predicted subgoal. We therefore provide extensive chunk-level qualitative examples in the main text and appendix; to save space, each panel reports only the final frame of an action chunk, which is the frame most aligned with the corresponding subgoal.

C Architecture and Implementation Details

C.1 Latent World Model Architecture

LaWM operates on features from the distilled DINOv3 ViT-B/16 encoder [17]. Its inverse-dynamics encoder and decoder are both 24-layer transformer modules. The encoder follows a V-JEPA2-style spatiotemporal design [46]: visual patches from the current and horizon observations are flattened into a single token sequence and processed jointly to infer the latent action posterior. Unlike the additive token injection used in Genie [16], our decoder conditions on the latent action z through adaptive layer normalization, which we found more stable in the cross-embodiment setting; additive injection can make fluctuations in the latent-action norm induce global shifts of visual tokens and cause sharp loss spikes. The auxiliary state predictor uses lightweight MLP heads to handle inconsistent state semantics across robot embodiments and is discarded after LaWM training.

C.2 Latent World Action Model Architecture

LaWAM follows the Qwen-GR00T architecture. We use the first 16 transformer layers of Qwen3-VL as the VLM backbone, and instantiate the action expert with four Alternate-DiT blocks, corresponding to 16 transformer layers in total. The hidden dimension is 1024. The policy input sequence contains the primary-view observation, the task instruction, latent-action query tokens, optional auxiliary-view images, and action-query tokens. All non-query tokens serve as context. With this ordering and a causal attention mask, the latent-action queries gather the information needed to drive LaWM, while the action queries still attend to the full semantic context used for control. The action expert receives the LaWM subgoal through Alternate-DiT [5]: instead of alternating only between visual and language streams, it alternates between the full VLM hidden state and a dynamics stream built from the current latent visual feature u and the predicted subgoal feature $\hat{u}_T$. All robot action labels are converted to an end-effector (EEF) representation. During policy training and evaluation, we do not provide proprioceptive state inputs, which helps avoid overfitting to trajectory-specific state traces and improves spatial generalization [47]. All experiments use RGB-only observations, with images resized to 256×256 .

C.3 Physical-Time Alignment for Mixed-Frequency Data

Robot datasets and downstream embodiments can operate at different control frequencies, so the same action-token index does not necessarily represent the same elapsed physical time. LaWAM handles this by keeping each dataset or embodiment branch at its native control frequency while defining every action chunk by a fixed physical interval τ . This makes the horizon correspond to a consistent amount of real elapsed time even when the number of discrete action tokens differs across branches. In the main text, T denotes a generic action-chunk horizon for readability; here we make its branch-specific form explicit. For a branch b with native control frequency h_b , the discrete action horizon is

$$H_b = \text{round}(\tau h_b), \quad (6)$$

Thus, the corresponding chunk contains $a_{b,1:H_b}$ and the paired horizon observation or feature is sampled at elapsed time τ . For example, under the same τ , a 5 Hz branch uses fewer action tokens than a 20 Hz branch, but both supervise LaWM against the future visual state reached after the same physical duration. This keeps latent subgoals comparable across datasets while allowing each branch to retain its native temporal resolution.

The action expert must also know where each action token lies in physical time. Because H_b differs across branches, the token index alone is ambiguous: index i may correspond to different elapsed times for different h_b . We therefore add a physical-time encoding to each action token. For token i from branch b , its timestamp is computed in seconds as $t_{b,i} = i/h_b$, and the sinusoidal encoding is

$$\phi(t_{b,i}) = \text{Concat} [\sin(t_{b,i}\omega_k), \cos(t_{b,i}\omega_k)]_{k=0}^{K-1}, \quad (7)$$

where $\omega_k = \exp\left(-\log(P_{\max}) \cdot \frac{k}{\max(K-1,1)}\right)$ and $K = \lceil d/2 \rceil$. We add $\phi(t_{b,i})$ to the corresponding action-query embedding before action denoising, so the expert receives both the token content and its physical-time coordinate. During batching, variable-length chunks are padded and masked, but the valid tokens keep their native timestamps. This maps mixed-frequency actions into a shared physical-time coordinate system: two tokens with the same elapsed time receive the same temporal code even if they occupy different indices, and two tokens with the same index receive different codes when their control frequencies imply different elapsed times. The resulting action sequence remains temporally aligned with the LaWM subgoal at τ . The controlled mixed-frequency benchmark construction is described in Sec. C.4.

C.4 Mixed-Frequency Training Experiment

This experiment isolates the role of physical-time encoding in mixed-source pretraining, where different control frequencies can make the same discrete action index correspond to different physical times. Directly quantifying this effect in the full pretraining mixture is difficult because dataset scale, embodiment, task distribution, camera setup, and language coverage vary simultaneously. We therefore construct a controlled LIBERO analysis and train LaWAM from scratch, so that control frequency is the primary manipulated factor.

Starting from the same native 20 Hz LIBERO trajectories, we create 10 Hz and 5 Hz versions by temporal downsampling, then co-train LaWAM on the combined 5/10/20 Hz data. This preserves the task distribution, embodiment, visual domain, and language instructions across branches while changing only the mapping between discrete action indices and elapsed physical time. Since the lower-frequency branches are derived from the same 20 Hz demonstrations, mixed-frequency training does not introduce additional expert trajectories; in this controlled setting, the native 20 Hz-only model therefore serves as an upper-bound reference rather than a weaker data baseline. The goal is to test whether physical-time encoding recovers performance close to this reference by resolving the control-frequency ambiguity introduced by mixed-frequency training.

As shown in Fig. 7, joint training without physical-time encoding substantially degrades success relative to the native 20 Hz upper-bound reference, while adding the encoding largely recovers performance toward this reference. This result supports our claim that physical-time encoding provides the temporal coordinate needed to make jointly trained mixed-frequency data consistent.

C.5 Training Details

LaWM training. For LaWM training, we use continuous latent actions and optimize Eq. 4 with 16 H100 GPUs for 100k steps using AdamW, learning rate 3×10^{-4} , weight decay 10^{-2} , and a global batch size of 1024. We set the KL regularization weight to $\beta = 10^{-5}$. The fixed physical-time horizon is 1.2 s for robot teleoperation videos and 0.4 s for egocentric human videos. Before DINOv3 encoding, we apply different random crops and color augmentations to encoder and decoder views, while keeping augmentations temporally consistent within each encoder clip; this discourages LaWM from memorizing embodiment-specific pixel layouts.

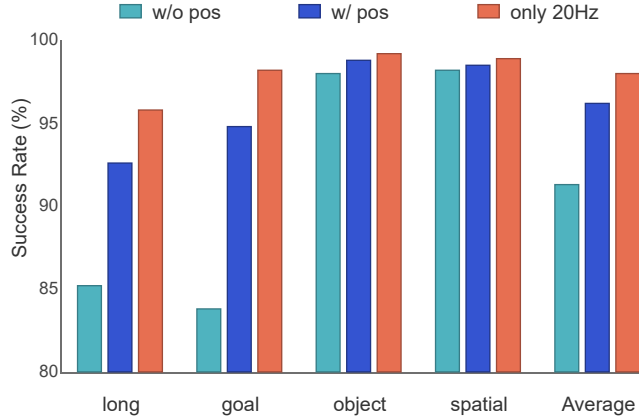


Figure 7: **Mixed-frequency training results on LIBERO.** Joint training across 5/10/20 Hz versions downsampled from the same LIBERO data shows that physical-time encoding resolves control-frequency confusion.

LaWAM policy training and evaluation. Stage-two policy integration uses robot trajectories with explicit language instructions. Egocentric human videos contribute through the dynamics prior learned by LaWM, but are not used for policy integration because they usually lack task descriptions that specify the intended robot behavior. A lightweight query-aggregation block maps the latent-action query to $\hat{z}$ before latent-action distillation. We set $\lambda_{\text{distill}} = \lambda_{\text{wm}} = 0.1$ in all experiments. Before benchmark-specific post-training, we run a 200k-step policy-integration pretraining stage for LaWAM on 64 H100 GPUs, using a global batch size of 1024. During this stage, the action expert is trained with a learning rate 10^{-4} , while all other modules use a learning rate 3×10^{-5} .

For post-training on each benchmark, we use a learning rate of 10^{-4} with cosine decay. Benchmark-specific training schedules and evaluation protocols are provided in the following subsections.

Unless otherwise specified, all policies are evaluated with 10 denoising steps. To measure inference latency, we run 1,000 repeated action-chunk predictions on an A100 GPU and report the average wall-clock latency. For WAM parameter counts reported in the paper, we exclude the video-diffusion VAE and the text encoder, whose size can reach 10B parameters. We compare against recent state-of-the-art VLA, latent-action, and WAM baselines, using numbers from the original papers whenever available and the strongest reproduced numbers otherwise.

D Experimental Details

D.1 LIBERO Protocol

We follow the standard LIBERO protocol [21] and train on four suites: LIBERO-Spatial, LIBERO-Object, LIBERO-Goal, and LIBERO-Long. Following the OpenVLA evaluation setup [2], we remove failed demonstrations from the training data. LaWAM is trained for 25k steps with a global batch size of 256. We report success rates over 2,000 trials across 40 tasks.

D.2 RoboTwin Protocol

We follow the multi-task training setup used by Motus [10] and LingBot-VA [12]: models are trained on a mixture of 2,500 demonstrations collected in clean scenes and 25,000 demonstrations collected under heavy scene randomization, spanning over 50 tasks. LaWAM is trained for 100k steps with a global batch size of 1024, requiring about 20 hours on 64 H100 GPUs. We report average success rates over 100 trials per task under both clean and randomized settings.

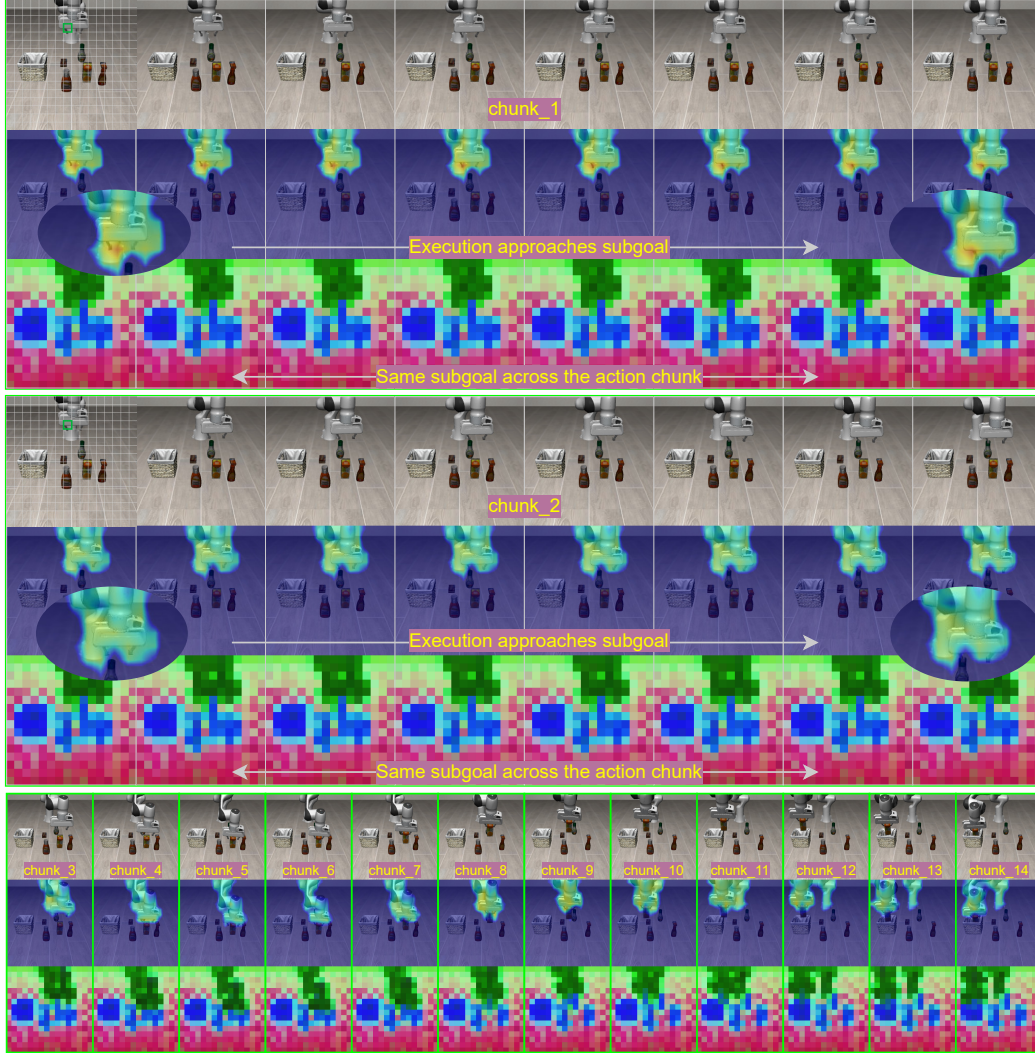


Figure 8: **Qualitative LIBERO examples showing LaWAM action execution together with latent world rollouts.** The predicted latent subgoals provide compact future dynamics that guide action generation without iterative pixel-space video prediction.

D.3 Real-World Protocol

We use two physical robot platforms, shown in Appendix Fig. 9. Pick-and-place and drawer opening are conducted on a Franka Emika Panda arm with a parallel gripper and an external RGB camera observing the workspace, while towel folding is conducted on a Quanta X1 bimanual robot. We train the policy with 150 real-world demonstrations for each Franka task and 280 demonstrations for towel folding, and evaluate each task over 30 real-world trials spanning both seen conditions and unseen test conditions. To ensure controlled comparison, the initial configurations used in each task are generated once and then fixed across LaWAM and all baselines, so that all methods are evaluated under identical physical conditions.

Pick-and-place. The robot must grasp a target object and place it into a bowl. The test objects include plush vegetable toys and toy ducks, covering irregular shapes and different surface properties; in particular, the duck is made of smooth plastic, which creates a more challenging contact condition for stable grasping, and the toy scallion is longer than the bowl diameter, requiring accurate placement despite the size mismatch. Across the object set, we evaluate 30 randomly generated initial configura-



(a) Franka setup.



(b) Quanta X1 setup.

Figure 9: **Real-world robot setups.** (a) A Franka Emika Panda robot with an external RGB camera is used for the single-arm pick-and-place and drawer-opening tasks. (b) The Quanta X1 bimanual robot is used for the long-horizon towel-folding task.

tions that vary the object location, bowl location, and object pose. Some test positions slightly extend beyond the spatial region covered by the training demonstrations, providing a controlled probe of spatial generalization. A qualitative pick-and-place execution with LaWM subgoal visualizations is shown in Appendix Fig. 11.

Drawer opening. The robot must grasp the handle of a closed drawer and pull it open. Due to the drawer material and sliding contact, pulling in a direction that is not parallel to the drawer motion creates substantial resistance, making the task sensitive to the grasp pose and pull direction. The drawer front edge is randomized within a predefined placement region, and the drawer is initialized with its orientation parallel to the short edge of the table before applying a random rotational perturbation of up to 15° . We evaluate 30 trials that uniformly cover 10 predefined initial configurations for this task. A representative drawer-opening rollout is visualized in Appendix Fig. 12.

Towel folding. The Quanta X1 operates on a deformable towel and must complete a long-horizon folding sequence. The task begins by shaking out and flattening the towel, then requires two long-edge folds followed by one short-edge fold. Even an experienced teleoperator requires about 70 s to complete the full sequence, highlighting the task’s long-horizon nature. This setting stresses bimanual coordination, contact-rich cloth manipulation, and temporal consistency over a substantially longer horizon than the rigid-object and articulated-object tasks. The full towel-folding sequence is shown in Appendix Fig. 13.

Failure inspection. For Fast-WAM, failures on the two Franka tasks are often associated with spatial perception near the target, such as inaccurate object-distance estimates before grasping in pick-and-place and handle-distance estimates in drawer opening. For LingBot-VA, towel-folding failures are closely tied to high inference latency: while the policy is generating the next action, the towel can continue moving and remain wrinkled, making it difficult to shake out folds promptly and causing the generated action to be conditioned on a stale visual state. LaWAM still shows a limitation in deformable-object manipulation, where the current LaWM feature resolution is not yet fine-grained enough to precisely capture subtle cloth deformations. Nevertheless, by accurately

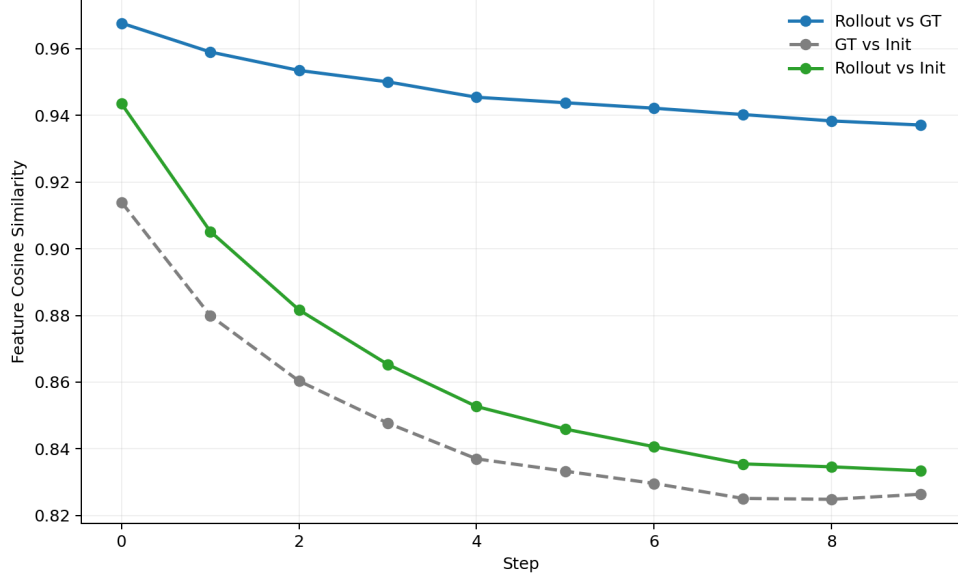


Figure 10: **Average LaWM rollout results over 500 LIBERO trajectories.** Curves show feature cosine similarity for rollout vs. ground-truth future states (blue), ground-truth future vs. initial states (gray), and rollout vs. initial states (green). The consistently high rollout–ground-truth similarity, together with the decreasing similarity to the initial state, indicates that LaWM follows the true latent dynamics rather than remaining close to the starting observation.

predicting robot-arm subgoals, LaWAM achieves the strongest towel-folding performance among the compared methods.

D.4 Additional LaWM Rollout Analysis

We use several complementary visualizations to examine the dynamics modeled by LaWM. Fig. 10 aggregates open-loop rollout behavior over 500 LIBERO trajectories by comparing the predicted horizon features against ground-truth future features and the initial features. The rollout remains close to the true future state while moving away from the initial state, indicating that LaWM models nontrivial latent dynamics rather than simply preserving the current observation. This aggregate probe supports LaWM’s dynamics-modeling ability over extended open-loop rollouts.

Figs. 14 and 15 provide additional cross-environment and cross-embodiment rollout examples. These visualizations complement Fig. 5: applying the same latent action in different visual contexts yields coherent but context-specific latent changes. We interpret this as evidence that LaWM uses the current latent visual state to ground latent actions in the embodiment and scene, while the latent action specifies the abstract transition to be realized.

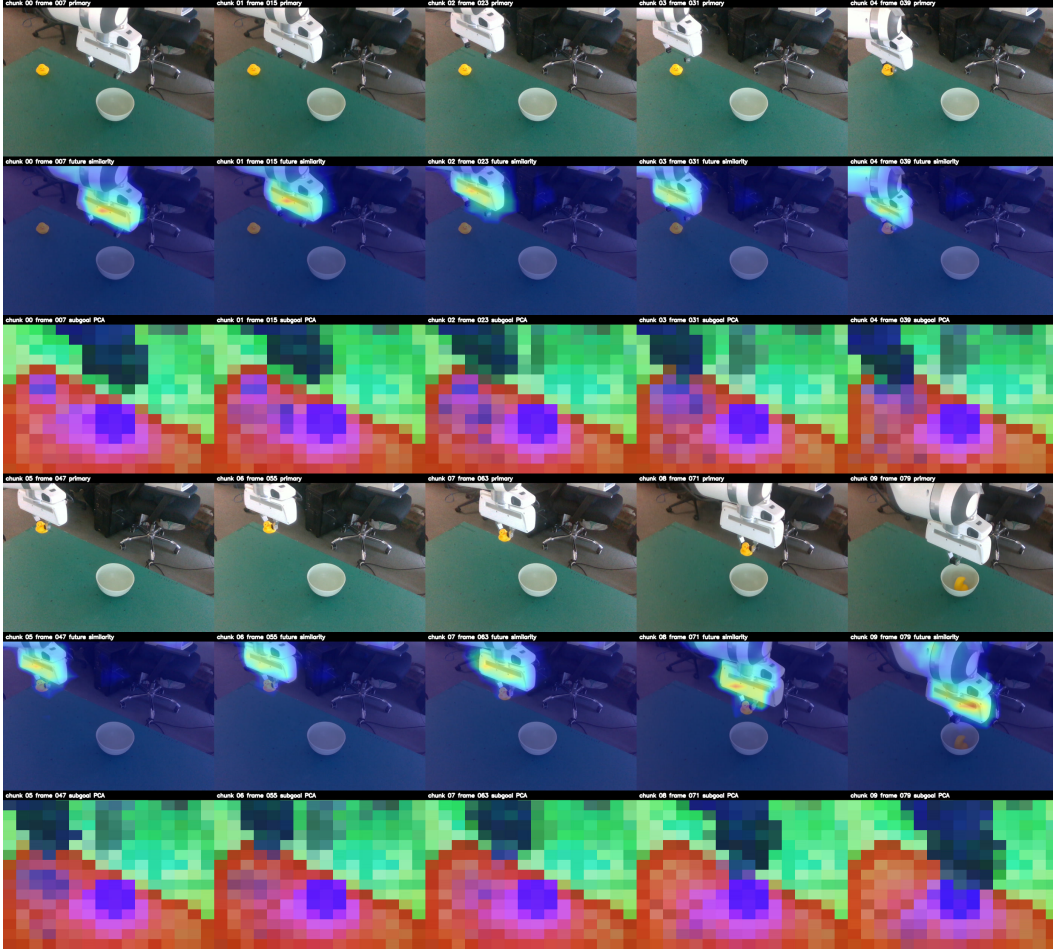


Figure 11: **Post-chunk observations for the real-world pick-and-place task with a toy duck on the Franka Emika Panda platform.** Each panel shows the observation after executing one action chunk, together with the future-similarity overlay and LaWM subgoal PCA. The overlap between the executed robot arm and the predicted subgoal heatmap indicates how closely the action expert follows the LaWM subgoal as it approaches, grasps, transports, and places the object into the bowl.

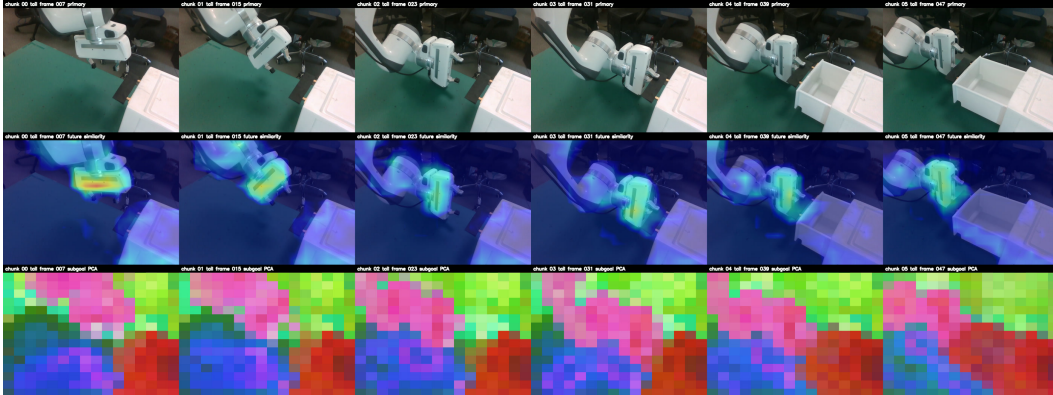


Figure 12: **Post-chunk observations for the real-world drawer-opening task on the Franka Emika Panda platform.** Each panel shows the observation after executing one action chunk, together with the future-similarity overlay and LaWM subgoal PCA. The overlap between the executed robot arm and the predicted subgoal heatmap indicates how closely the action expert follows the LaWM subgoal while grasping the handle and pulling the drawer open.

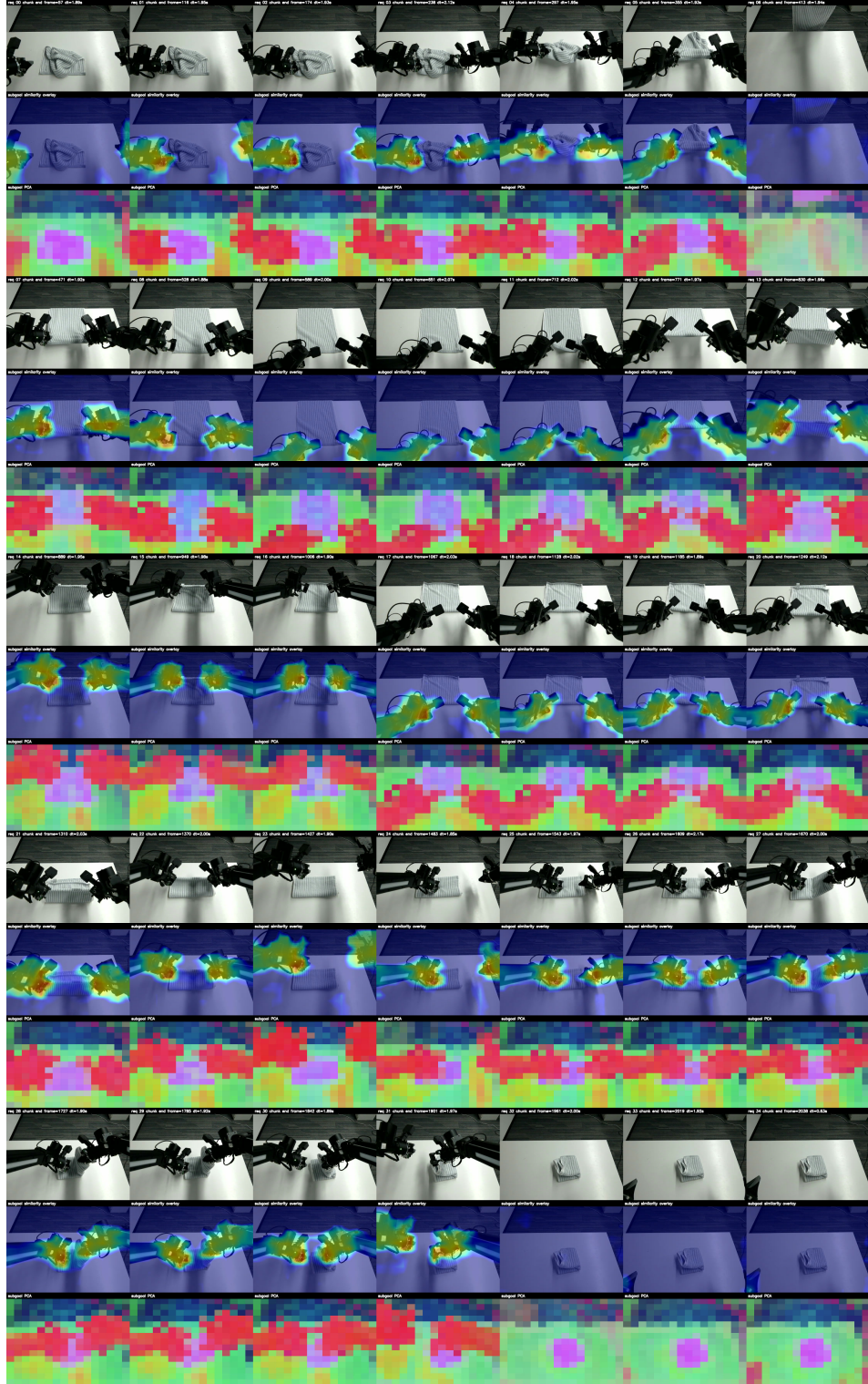


Figure 13: **Post-chunk observations for the real-world towel-folding task on the Quanta X1 platform.** The robot first shakes out and flattens the towel, then completes two long-edge folds followed by one short-edge fold, testing long-horizon bimanual deformable-object manipulation. Each panel shows the observation after executing one action chunk; the overlap between the executed robot arm and the predicted subgoal heatmap indicates how closely the action expert follows the LaWM subgoal.

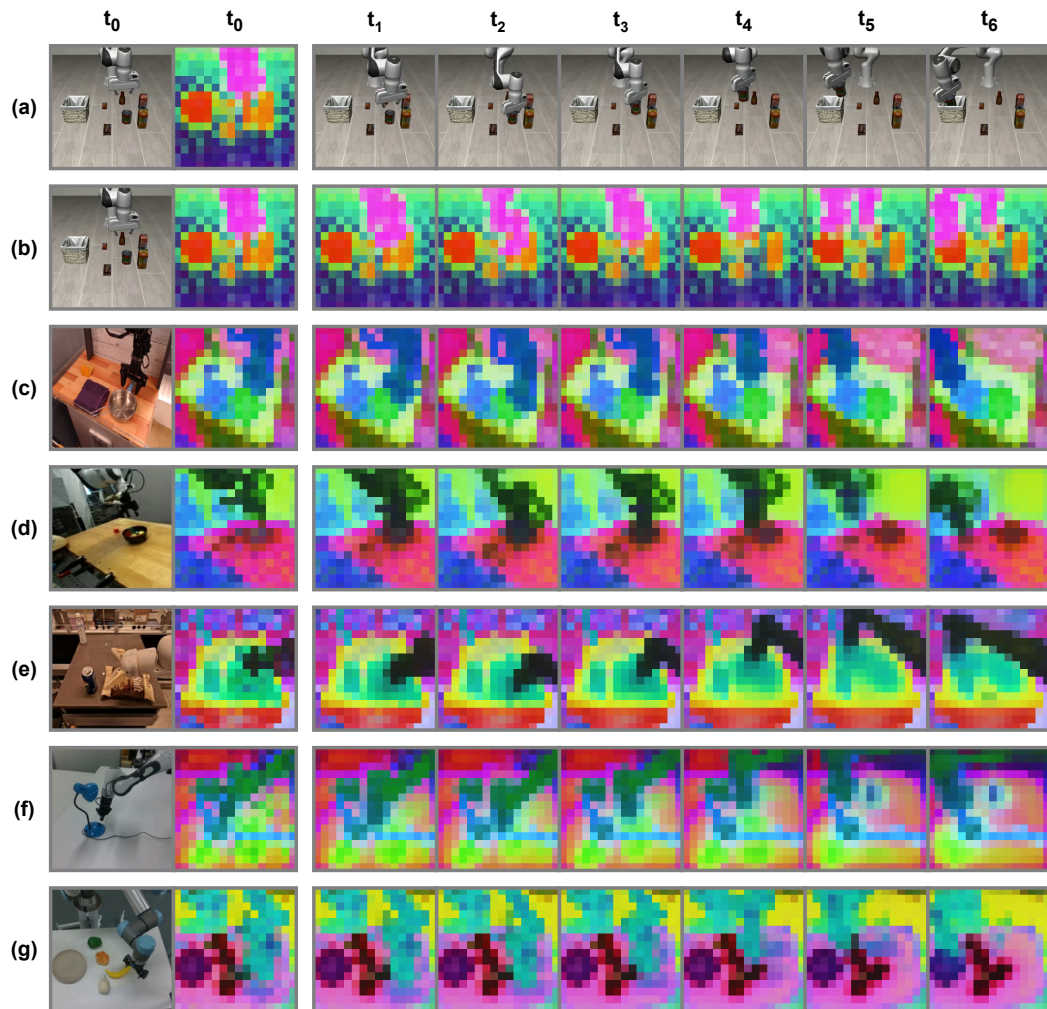


Figure 14: Single-arm cross-embodiment rollout.



Figure 15: **Dual-arm cross-embodiment rollout.** Panels (e)–(j) show unseen scenes; panels (f)–(j) use screenshots from *pi.website*.